

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 10_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 47.5

Section 1 : Coding

1. Problem Statement

Jovika is learning about heaps and wants to implement a program that constructs a max heap and inserts elements into it.

She wants to write a program that implements max heap insertion. The program defines functions to swap elements and perform maximum heapification. She wants to extend this program to take a list of integers, build a max heap, and insert a new element into the heap.

Help her in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements to be inserted into the max heap.

The second line consists of N space-separated integers, representing the elements to be inserted into the max heap.

The third line consists of an integer representing the new element to be inserted into the max heap.

Output Format

The output prints the space-separated integers, representing the elements in the max heap after inserting the new element in the order of insertion.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

3 1 4 2 5

6

Output: 6 3 5 2 1 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {
```

```
    int t = *a;
```

```
    *a = *b;
```

```
    *b = t;
```

```
}
```

```
void heapifyUp(int arr[], int i) {
```

```
    while (i > 0 && arr[(i - 1) / 2] < arr[i]) {
```

```
        swap(&arr[i], &arr[(i - 1) / 2]);
```

```
        i = (i - 1) / 2;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```

int arr[50];

for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
    heapifyUp(arr, i);
}

int x;
scanf("%d", &x);

arr[n] = x;
heapifyUp(arr, n);
n++;

for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);

return 0;
}

```

Status : Partially correct

Marks : 7.5/10

2. Problem Statement

Bob is fascinated by numbers and loves arranging them in various ways. His recent exploration led him to Max Heaps.

Bob likes odd numbers more than even numbers. After inserting elements into a Max Heap, he wants to remove all even numbers from it while keeping the Max Heap property intact.

Your task is to help Bob build a Max Heap from the given elements, display it, delete all the even numbers from the resultant array and then rearrange the elements in the form of Max Heap.

Input Format

The first line of input consists of an integer N, representing the number of elements initially to be inserted into the Max Heap.

The second line consists of N space-separated integers, representing the elements to be inserted into the heap.

Output Format

The first line of output prints the space-separated elements of the Max Heap after building it from the given N elements.

The second line prints the space-separated elements of the Max Heap after removing even elements.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3 2 1

Output: 3 2 1

3 1

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {  
    int t = *a;  
    *a = *b;  
    *b = t;  
}
```

```
void heapify(int arr[], int n, int i) {  
    int largest = i, l = 2*i+1, r = 2*i+2;
```

```
    if (l < n && arr[l] > arr[largest])  
        largest = l;
```

```
    if (r < n && arr[r] > arr[largest])  
        largest = r;
```

```
    if (largest != i) {  
        swap(&arr[i], &arr[largest]);
```

```

        heapify(arr, n, largest);
    }
}

void buildHeap(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[50];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    buildHeap(arr, n);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");

    int temp[50], k = 0;
    for (int i = 0; i < n; i++)
        if (arr[i] % 2 != 0)
            temp[k++] = arr[i];

    buildHeap(temp, k);

    for (int i = 0; i < k; i++)
        printf("%d ", temp[i]);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Iniya is studying data structures and algorithms, and she is currently learning about heaps.

She wants to practice implementing Min-Heap and perform a deletion operation on it. A min-heap is a binary tree in which the value of each node is less than or equal to the values of its children.

Write a program to assist Iniya in building a min-heap and delete the root element.

Input Format

The first line of input consists of an integer N, representing the number of elements in the initial Min-Heap.

The second line consists of N space-separated integers, where each integer represents an element in the Min-Heap.

Output Format

The output prints the space-separated elements of the Min-Heap after performing the deletion operation of the root.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

80 40 30 10 50 33 66

Output: 30 40 33 80 50 66

Answer

```
// You are using GCC
#include <stdio.h>
```

```
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
```

```

    *b = t;
}

void heapify(int arr[], int n, int i) {
    int smallest = i, l = 2*i+1, r = 2*i+2;

    if (l < n && arr[l] < arr[smallest])
        smallest = l;

    if (r < n && arr[r] < arr[smallest])
        smallest = r;

    if (smallest != i) {
        swap(&arr[i], &arr[smallest]);
        heapify(arr, n, smallest);
    }
}

```

```

void buildHeap(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
}

```

```

int main() {
    int n;
    scanf("%d", &n);

    int arr[50];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    buildHeap(arr, n);

    arr[0] = arr[n-1];
    n--;

    heapify(arr, n, 0);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

A trie (pronounced as "try") or prefix tree is a tree data structure used to store and retrieve keys in a dataset of strings efficiently. This data structure has various applications, such as autocomplete and spellchecker.

Implement the Trie class:

Trie() Initializes the trie object.
void insert(String word): Inserts the string word into the trie.
boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
boolean startsWith(String prefix) Returns true if a previously inserted string word has the prefix, and false otherwise.

Example

Input:

4

apple

cat

orange

mat

cat

ora

Output:

The word is present in the trie.

There is a word with the given prefix in the trie.

Input Format

The first line of the input consists of the number of words to insert, N.

The next N lines of input consist of the words in each line.

The next line of input consists of the word to search.

The last line of the input consists of the prefix to search.

Output Format

When searching for a word using the search method, the program will output one of the following messages:

- If the word is present in the trie, the output prints "The word is present in the trie."
- If the word is not present in the trie, the output prints "The word is not present in the trie."

When checking for a prefix using the startsWith method, the program will output one of the following messages:

- If there is a word in the trie with the given prefix, the output prints "There is a word with the given prefix in the trie."
- If there is no word in the trie with the given prefix, the output prints "There is no word with the given prefix in the trie."

Sample Test Case

Input: 4

apple

cat

orange

mat

catlyn

ora

Output: The word is not present in the trie.

There is a word with the given prefix in the trie.

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>

struct Trie {
    struct Trie* child[26];
    int isEnd;
};

struct Trie* createNode() {
    struct Trie* node = (struct Trie*)malloc(sizeof(struct Trie));
    for (int i = 0; i < 26; i++)
        node->child[i] = NULL;
    node->isEnd = 0;
    return node;
}

void insert(struct Trie* root, char* word) {
    struct Trie* temp = root;
    for (int i = 0; word[i]; i++) {
        int idx = word[i] - 'a';
        if (!temp->child[idx])
            temp->child[idx] = createNode();
        temp = temp->child[idx];
    }
    temp->isEnd = 1;
}

int search(struct Trie* root, char* word) {
    struct Trie* temp = root;
    for (int i = 0; word[i]; i++) {
        int idx = word[i] - 'a';
        if (!temp->child[idx])
            return 0;
        temp = temp->child[idx];
    }
    return temp->isEnd;
}

int startsWith(struct Trie* root, char* prefix) {
    struct Trie* temp = root;
    for (int i = 0; prefix[i]; i++) {
        int idx = prefix[i] - 'a';

```

```

    if (!temp->child[idx])
        return 0;
    temp = temp->child[idx];
}
return 1;
}

int main() {
    int n;
    scanf("%d", &n);

    struct Trie* root = createNode();

    char word[50];
    for (int i = 0; i < n; i++) {
        scanf("%s", word);
        insert(root, word);
    }

    char searchWord[50], prefix[50];
    scanf("%s", searchWord);
    scanf("%s", prefix);

    if (search(root, searchWord))
        printf("The word is present in the trie.\n");
    else
        printf("The word is not present in the trie.\n");

    if (startsWith(root, prefix))
        printf("There is a word with the given prefix in the trie.");
    else
        printf("There is no word with the given prefix in the trie.");

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Given an array of integers representing the combat power of N players.

You need to efficiently perform the following operations:

Query (Q L R): Find the maximum combat power between ranks L and R (inclusive). Update (U i val): Update the power of the player at index i to a new value val.

To ensure efficiency, implement these operations using a Segment Tree.

Input Format

The first line contains two space-separated integers, N and Q, representing the number of players and the number of operations.

The second line contains N space-separated integers representing the initial combat power scores of all players, where index 0 is the top-ranked player.

The next Q lines describe the operations:

- "Q L R" Query represents the maximum combat power from ranks L to R (inclusive).
- "U i val" Updates the power score of the player at rank i to val.

All indices are 0-based.

Output Format

For each query Q L R, print the maximum combat power in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
4 7 2 9 5 1
Q 1 4
U 3 6
Q 1 4
U 0 10

Q 0 2

Output: 9

7

10

Answer

// You are using GCC

#include <stdio.h>

int tree[100], arr[50];

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
void build(int node, int start, int end) {  
    if (start == end) {  
        tree[node] = arr[start];  
    } else {  
        int mid = (start + end) / 2;  
        build(2*node, start, mid);  
        build(2*node+1, mid+1, end);  
        tree[node] = max(tree[2*node], tree[2*node+1]);  
    }  
}
```

```
void update(int node, int start, int end, int idx, int val) {  
    if (start == end) {  
        arr[idx] = val;  
        tree[node] = val;  
    } else {  
        int mid = (start + end) / 2;  
        if (idx <= mid)  
            update(2*node, start, mid, idx, val);  
        else  
            update(2*node+1, mid+1, end, idx, val);  
  
        tree[node] = max(tree[2*node], tree[2*node+1]);  
    }  
}
```

```
int query(int node, int start, int end, int l, int r) {
```

```
if (r < start || end < l)
    return -1;
```

```
if (l <= start && end <= r)
    return tree[node];
```

```
int mid = (start + end) / 2;
int p1 = query(2*node, start, mid, l, r);
int p2 = query(2*node+1, mid+1, end, l, r);
```

```
if (p1 == -1) return p2;
if (p2 == -1) return p1;
return max(p1, p2);
```

```
}
```

```
int main() {
    int n, q;
    scanf("%d %d", &n, &q);
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
```

```
    build(1, 0, n-1);
```

```
    while (q--) {
        char type;
        scanf(" %c", &type);
```

```
        if (type == 'Q') {
            int l, r;
            scanf("%d %d", &l, &r);
            printf("%d\n", query(1, 0, n-1, l, r));
        } else {
            int i, val;
            scanf("%d %d", &i, &val);
            update(1, 0, n-1, i, val);
        }
    }
```

```
}
```

```
return 0;
```

```
}
```

Status : Correct

Marks : 10/10